UNIT_II

FILES: File Objects, File Built-in Function [open()], File Built-in Methods, File Built-in Attributes, Standard Files, Command-line Arguments, File System, File Execution, Persistent Storage Modules, Related Modules

1. Introduction to Files in Python

- A file in computing is a collection of data stored in a storage device, with a name and extension (e.g., data.txt, script.py).
- In Python, files are used to store data permanently, unlike variables whose values disappear when the program ends.
  To work with files in Python, you typically:

  Open a file

  Read or write data

  Close the file

2. File Objects

A **file object** is an instance of Python's built-in file class returned by the open() function. It acts as a **bridge** between your Python program and the actual file stored on disk.

open() #function syntax

file_object = open(file_name, mode)

Parameters:

- file_name: The name (and path) of the file to open.
- mode: A string that specifies the operation mode.

3. File Opening Modes

Mode	Meaning
'r'	Read (default) – file must exist
'w'	Write – creates a file or overwrites existing
'x'	Create – creates a new file, errors if file exists
'a'	Append – adds data to end of file without overwriting
'b'	Binary mode (e.g., 'rb', 'wb')
't'	Text mode (default)
'r+'	Read and write
'w+'	Write and read (overwrites)
'a+'	Append and read

Example:

```
f = open("example.txt", "r")  # read mode
```

4. File Object Attributes

Once you have a file object, you can check its properties:

Attribute	Description
.name	Name of the file
.mode	Mode in which the file was opened
.closed	Returns True if the file is closed
.readable()	Checks if file can be read
.writable()	Checks if file can be written

Example

```
f = open("example.txt", "w")
print(f.name)      # example.txt
print(f.mode)      # w
print(f.closed)    # False
f.close()
print(f.closed)    # True
```

Example

```
# Example: Checking if a file is readable
file = open(“example.txt", "r")  # Open in read mode

if file.readable():
    print("The file can be read.")
    content = file.read()  # Reading the file content
    print("File content:")
    print(content)
else:
    print("The file cannot be read.")

file.close()
```

Example

```
# Example: Checking if a file is writable
file = open("output.txt", "w")  # Open in write mode

if file.writable():
print("The file can be written to.")
file.write("Hello, this is a test message.\n")
else:
print(" The file cannot be written to.")

file.close()
```

5. Reading from a File

Python provides several ways to read:

a) .read()

Reads the entire file or a specified number of characters.

```
f = open(“example.txt", "r")
content = f.read()
print(content)
f.close()
```

b) .readline()
Reads a single line.
line1 = f.readline()

```
# Create a sample text file
with open("sample.txt", "w") as file:
    file.write("First line\n")
    file.write("Second line\n")
    file.write("Third line\n")

# Open the file for reading
with open("sample.txt", "r") as file:
    line1 = file.readline()   # Reads first line
    print("Line 1:", line1.strip())

    line2 = file.readline()   # Reads second line
    print("Line 2:", line2.strip())

    line3 = file.readline()   # Reads third line
    print("Line 3:", line3.strip())

    line4 = file.readline()   # Tries to read next line (empty if end of file)
    print("Line 4:", line4.strip())
```

c) .readlines()

Reads all lines into a list.

```
# Create a sample text file
with open("sample.txt", "w") as file:
    file.write("First line\n")
    file.write("Second line\n")
    file.write("Third line\n")

# Open the file for reading
with open("sample.txt", "r") as file:
    all_lines = file.readlines()   # Reads all lines into a list
    print(all_lines)               # Prints list of lines

# Accessing lines individually
print("Line 1:", all_lines[0].strip())
print("Line 2:", all_lines[1].strip())
print("Line 3:", all_lines[2].strip())
```

strip method

In Python, **.strip()** is a string method that removes **leading and trailing whitespace** (including spaces, tabs \t, and newline characters \n) from a string.

```
text = "   Hello World!   \n"
print("Before strip:", repr(text))

cleaned_text = text.strip()
print("After strip:", repr(cleaned_text))
```

- **Output**

```
Before strip: '   Hello World!   \n'
After strip: 'Hello World!'
```

```
text = "Hello\nWorld\ngames"
print(text)         # User-friendly output
print(repr(text))   # Developer/debug output
```

6. Writing to a File

- Writing overwrites existing data unless opened in append mode.

```
f = open("output.txt", "w")
f.write("Hello World\n")
f.write("This is a test.")
f.close()
```

Writing multiple lines:

```
lines = ["First line\n", "Second line\n", "Third line\n"]
f = open("output.txt", "w")
f.writelines(lines)
f.close()
```

7. File Pointer Operations

- The file pointer indicates the current position in the file.
- **f.tell()** → Returns the current position of file pointer.
- **f.seek(offset, whence)** → Moves the pointer.
  - whence=0: start of file (default)
  - whence=1: current position
  - whence=2: end of file

Example

```
f = open("data.txt", "r")   # Open file in read mode
print(f.tell())             # Position of file pointer: 0 (start of file)

f.read(5)                   # Read first 5 characters from the file
print(f.tell())             # Position is now 5 (we've moved forward 5 chars)

f.seek(0)                   # Move file pointer back to the start (offset 0)
f.close()                   # Close the file
```

- **tell()** → returns the current position of the file pointer in bytes.
- **read(n)** → reads n characters (or bytes if in binary mode) from the current pointer location.
- **seek(offset)** → moves the file pointer to the given position from the start of the file.
- seek(0) → moves to the very beginning.
- seek(10) → moves to the 10th byte.
- **close()** → closes the file and releases resources.

Think of the file as a movie tape:

- tell() → tells you **where the playhead is**.
- read() → plays forward and moves the playhead.
- seek() → jumps the playhead to a specific frame.

8. Closing a File

Always close a file after operations:

f.close()

Or better, use **with** statement (auto-closes the file):

```
with open("data.txt", "r") as f:
    content = f.read()
# file auto-closed here
```

Example – Reading and Writing

```
# Write
with open("demo.txt", "w") as f:
    f.write("Python File Handling Example\n")
    f.write("Line 2\n")
print(f.closed)

# Read
with open("demo.txt", "r") as f:
    for line in f:
        print(line.strip())
```

File Built-in Methods in Python

1. File Information & Position

Method	Purpose	Example
file.name	Returns the file name	print(f.name)
file.mode	Returns the mode in which file is opened	print(f.mode)
file.closed	Returns True if file is closed	print(f.closed)
file.tell()	Returns current file position (cursor)	print(f.tell())
file.seek(pos)	Moves cursor to a specific position	f.seek(0)

2. Reading Data

Method	Purpose	Example
file.read(size)	Reads size characters (or entire file if omitted)	f.read(10)
file.readline(size)	Reads one line from file	f.readline()
file.readlines(hint)	Reads all lines into a list	lines = f.readlines()

3. Writing Data

Method	Purpose	Example
file.write(str)	Writes a string to file	f.write("Hello")
file.writelines(list)	Writes multiple strings (no newline added automatically)	f.writelines(["a\n","b\n"])

4. Checking File Permissions

Method	Purpose	Example
file.readable()	Returns True if file can be read	f.readable()
file.writable()	Returns True if file can be written	f.writable()

5. Closing Files

Method	Purpose	Example
file.close()	Closes file	f.close()

File Built-in Attributes

- In Python, **file objects** (the ones you get when you use open()) have **built-in attributes** that store information about the file and its current state.
  These attributes are **not methods**—they don't do an action, they just hold values describing the file object.
- Here's a **detailed breakdown** with examples:

1. file.name

- **Meaning** → The name of the file you opened (string).
- **Purpose** → Lets you check or confirm which file is associated with this file object.
- **Example**:

```
f = open("example.txt", "r")
print(f.name)  # Output: example.txt
f.close()
```

2. file.mode

- **Meaning** → Shows the mode in which the file was opened ("r", "w", "a", "rb", etc.).
- **Purpose** → Useful for verifying if the file is in the right mode before performing operations.
- **Example**:

```
f = open("example.txt", "rb")
print(f.mode)  # Output: rb
f.close()
```

3. file.closed

- **Meaning** → Boolean value (True or False) indicating whether the file is closed.
- **Purpose** → Lets you check if a file is still open before performing operations to avoid errors.
- **Example**:

```
f = open("example.txt", "r")
print(f.closed)  # Output: False
f.close()
print(f.closed)  # Output: True
```

4. file.encoding

- **Meaning** → The name of the encoding used for the file (e.g., "UTF-8") if it’s a text file.
- **Purpose** → Helps you ensure correct reading/writing of special characters.
- **Note** → For binary files ("rb"), this will be None.
- **Example**:

```
f = open("example.txt", "r", encoding="utf-8")
print(f.encoding)  # Output: UTF-8
f.close()
```

5. file.newlines

- **Meaning** → Shows the newline characters (\n, \r, \r\n) encountered in the file so far.
- **Purpose** → Useful if working with files from different operating systems (Windows, Mac, Linux).
- **Example**:

```
f = open("example.txt", "r")
f.read()
print(f.newlines)  # Output could be '\n' or '\r\n'
f.close()
```

6. file.buffer

- **Meaning** → For text files, this gives you the underlying binary buffer object.
- **Purpose** → Lets you access raw bytes even if the file was opened in text mode.
- **Example**:

```
f = open("example.txt", "r")
print(f.buffer)  # Output: <_io.BufferedReader ...>
f.close()
```

Summary Table

Attribute	Description	Example Value
name	Name of the file	"example.txt"
mode	Mode in which file is opened	"r", "wb"
closed	Boolean indicating if file is closed	True / False
encoding	Encoding used (for text files)	"UTF-8"
newlines	Newline characters seen in file	"\n", "\r\n"
buffer	Underlying buffer for binary access	<_io.BufferedReader>

Standard Files

In Python, **standard files** refer to three default file streams that are automatically opened when your program starts.
These are part of the **standard input/output system** that most programming languages use.

These are called **"standard streams"** because they are *already open* when your program starts — you don’t need to call open().

1. Standard Input (sys.stdin)

- **Purpose:** Reads data from the keyboard or another input stream.
- **Default:** Keyboard.
- **Mode:** Read-only.
- **Example:**

```
import sys
name = sys.stdin.readline()
print(f"Hello, {name.strip()}!")
#When you run this, Python waits for you to type something and press Enter.
```

2. Standard Output (sys.stdout)

- **Purpose:** Sends output to the console.
- **Default:** Console screen.
- **Mode:** Write-only.
- **Example:**

```
import sys
sys.stdout.write("This is standard output.\n")
print("This is also standard output (via print).")
```

3. Standard Error (sys.stderr)

- **Purpose:** Displays error messages and diagnostics.
- **Default:** Console screen (but separate from stdout).
- **Mode:** Write-only.
- **Example:**

```
import sys
sys.stderr.write("This is an error message!\n")
```

Key Points

Standard File	Python Object	Purpose	Default Destination
Standard Input	sys.stdin	Read input from user or program	Keyboard
Standard Output	sys.stdout	Display normal program output	Screen
Standard Error	sys.stderr	Display error messages and diagnostics	Screen

Redirection

```
import sys

# Redirect stdout to a file
sys.stdout = open("output.txt", "w")
print("This will go to the file, not the console.")

# Redirect stderr to a file
sys.stderr = open("errors.txt", "w")
sys.stderr.write("Error message stored in errors.txt")
```

Command-line Arguments

In Python, **command-line arguments** are values you pass to a Python script when you run it from the terminal or command prompt. They let you provide inputs to your program without hardcoding them.

What are Command-Line Arguments?

When you normally run a Python program, you do something like:

python myprogram.py

But we can also pass **extra information (arguments)** from the terminal:

python myprogram.py hello world 123

Inside the Python program, we can access these arguments using the sys module.

import sys

print("Program name:", sys.argv[0]) # The filename

print("First argument:", sys.argv[1])

print("Second argument:", sys.argv[2])

If we run:

python myprogram.py apple banana

Output:

Program name: myprogram.py

First argument: apple

Second argument: banana

Understanding sys.argv

sys.argv is just a **list** (like Python lists).
It stores everything you type after python.
Example:
python test.py red green blue
Then in Python:
sys.argv = ["test.py", "red", "green", "blue"]
sys.argv[0] → "test.py" (always the program name)
sys.argv[1]→"red"
sys.argv[2]→"green"
sys.argv[3] →"blue"

Example: Adding Numbers from Command Line

```
import sys
num1 = int(sys.argv[1])   # First number
num2 = int(sys.argv[2])   # Second number
print("Sum:", num1 + num2)
```

Run in terminal:

```
python add.py 5 10
```

Output:

```
Sum: 15
```

File System

A **File System** is a way for an operating system (Windows, Linux, macOS, etc.) to **organize, store, and manage files** on a storage device (like a hard disk, SSD, USB, or CD).

Think of it like a **giant filing cabinet**:

- Cabinet → The **storage device** (C: drive, D: drive, USB drive)
- Drawers → **Folders/Directories**
- Papers → **Files** (text, images, programs, etc.)
- Labels → **File names and extensions** (.txt, .py, .jpg)

Without a file system, the computer would see only raw 1s and 0s — impossible for humans to use.

Main Responsibilities of a File System

- **File Organization** – Keeps files in directories (folders).
- **File Naming** – Every file has a name and extension (hello.py, song.mp3).
- **File Access** – Defines how files are opened, read, written, or deleted.
- **File Metadata** – Stores information about a file (size, creation date, permissions).
- **Security & Permissions** – Controls who can read/write/execute a file.
- **Storage Management** – Decides where on the disk each file's data is stored.

Examples of File Systems

- **FAT32** – Old, used in USB drives, but limited (max file size: 4GB).
- **NTFS** – Modern Windows file system, supports large files, security permissions.
- **ext4** – Linux file system, very reliable.
- **HFS+ / APFS** – macOS file systems.

How Files are Structured

Every file has **two parts**:

- **File name** → Human-readable (resume.docx, photo.png)
- **File content** → Actual data (text, music, image pixels, etc.)

Example:

resume.docx → File name

01001100101… → Stored binary data inside the file

File System Operations in Python

1. **Creating Files**

Using open()

If the file does not exist, Python will create it when opened in write (w) or append (a) mode.

```
# Create a new file
f = open("myfile.txt", "w")
f.write("Hello, this is a new file!\n")
f.close()
```

File "myfile.txt" will be created in the current working directory.

Creating Directories

To create folders/directories:

```
import os
# Create a single directory
os.mkdir("myfolder")
# Create nested directories
os.makedirs("myfolder/subfolder/nested")
```

os.mkdir() → creates **one folder**.
os.makedirs() → creates **nested folders** (parents included).

Listing Directory Contents

To see what files/folders exist in a directory:

```
import os
print(os.listdir(".")) # List current directory
print(os.listdir("myfolder")) # List inside 'myfolder'
```

os.listdir(path) → returns a list of all files and directories inside.

Walking Through Directories

To **traverse** (walk) through all files and subfolders:

- import os
- for root, dirs, files in os.walk("."):
- print("Current Folder:", root)
- print("Subfolders:", dirs)
- print("Files:", files)
- print("-" * 30)
- break #-- stops after first folder only

5. Checking File/Directory Existence

```
import os

print(os.path.exists("myfile.txt"))    # True if file/folder exists

print(os.path.isfile("myfile.txt"))    # True if it's a file

print(os.path.isdir("myfolder"))       # True if it's a folder
```

6. Deleting Files and Directories

```
import os
import shutil

# Delete a file
os.remove("myfile.txt")

# Delete an empty directory
os.rmdir("myfolder")

# Delete a directory with contents
shutil.rmtree("myfolder")
```

shutil.rmtree() deletes everything inside without asking.

7. **Renaming and Moving**

```
import os


# Rename a file
os.rename("old.txt", "new.txt")


# Move a file (also renames if new path is given)
os.rename("new.txt", "myfolder/new.txt")
```

8. Using Pathlib (Modern & Easier)

```
from pathlib import Path

# Create a Path object
p = Path("example.txt")

# Create a file
p.write_text("Hello with pathlib!")

# Read file content
print(p.read_text())

# Create directory
Path("newfolder").mkdir(exist_ok=True)
```

Path("newfolder") → Creates a Path object pointing to a folder named newfolder in the current directory.
mkdir() → Creates the directory.
exist_ok=True → Prevents an error if the folder already exists.

- If exist_ok=False (default), Python will raise a FileExistsError if the folder already exists.
- With exist_ok=True, it will silently skip creation if the folder is already present.”’

```
# List files in a folder
for f in Path(".").iterdir():
print(f)
```

File Execution

- When we say "execute a file", it usually means **running a Python program/script** stored in a file.
- Example: If you have a file myprogram.py with this code:

```
print("Hello, world!")
```

You can **execute it** (run it) from the command line:

```
python myprogram.py
```

This tells Python to read the file myprogram.py line by line and execute the instructions.

Ways to Execute a File

1. **From the Command Line**

python script.py

Runs the file directly.

2. **From Another Python Script**

You can execute one Python file from another:

```
# main.py
exec(open("myprogram.py").read())
```

This reads the code inside myprogram.py and executes it.

3. **Using import**

Suppose you have a file **myprogram.py**:

```
# myprogram.py
def greet(name):
    return f"Hello, {name}!"
```

Now in another file:

```
import myprogram
print(myprogram.greet("Neelkanth"))
```

4. **Using subprocess Module**

This runs the file as if from the terminal:

```
import subprocess
subprocess.run(["python", "myprogram.py"])
```

What is Persistent Storage?

Persistent storage means **data that is saved beyond the lifetime of a program execution**. Unlike variables in memory (RAM), persistent storage keeps data stored on disk (files, databases, etc.) so you can retrieve it later.

Python Modules for Persistent Storage

1. **Pickle**

- Converts Python objects into a byte stream (serialization) and back (deserialization).
- Used for saving objects directly.

```
import pickle

data = {"name": "Neelkanth", "age": 52}

# Save to file
with open("data.pkl", "wb") as f:
    pickle.dump(data, f)

# Load back
with open("data.pkl", "rb") as f:
    loaded = pickle.load(f)
print(loaded)
```

2. **Shelve**

- Works like a **persistent dictionary** stored in a file.
- Keys must be strings, values can be any picklable Python object.

```
import shelve

with shelve.open("mydata") as db:
    db["user"] = {"name": "Neelkanth", "age": 25}

with shelve.open("mydata") as db:
    print(db["user"])   # {'name': 'Neelkanth', 'age': 25}
```

3. **dbm (Database Manager)**

- Key-value store where keys & values are **bytes**.
- Lightweight database, useful for simple storage.

```
import dbm

with dbm.open("store", "c") as db:
    db[b"name"] = b"Neelkanth"

with dbm.open("store", "r") as db:
    print(db[b"name"])   # b'Neelkanth'
```

4. json

- Stores data in **human-readable format** (text-based).
- Great for sharing data across programs.

```
import json

data = {"city": "Delhi", "population": 32000000}

# Save
with open("data.json", "w") as f:
    json.dump(data, f)

# Load
with open("data.json", "r") as f:
    loaded = json.load(f)
print(loaded)
```

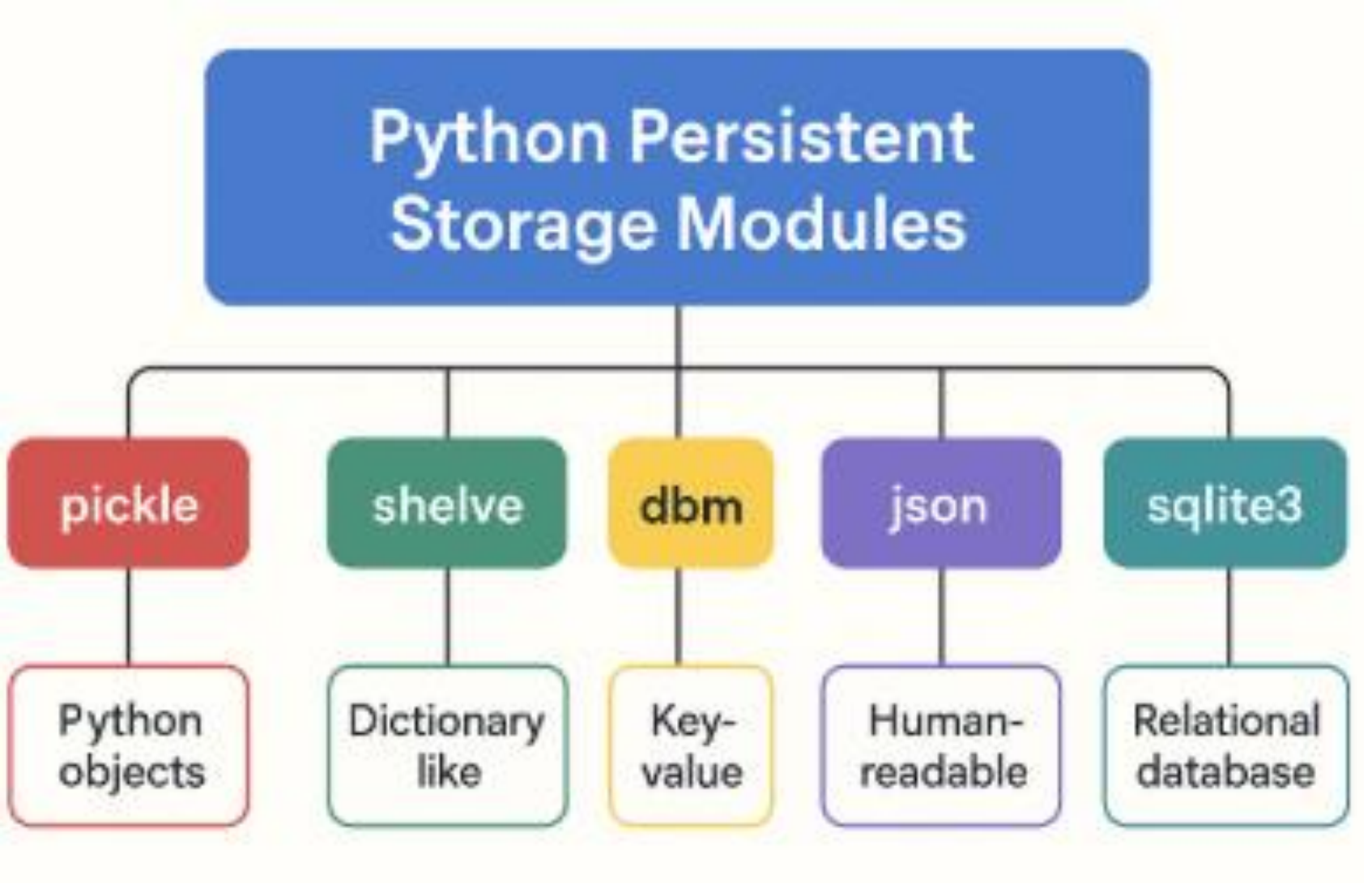
5. **SQLite (sqlite3 module)**

- Embedded SQL database stored in a single file.
- More powerful for structured queries.

```
import sqlite3

conn = sqlite3.connect("data.db")
cursor = conn.cursor()

cursor.execute("CREATE TABLE IF NOT EXISTS users (name TEXT, age INTEGER)")
cursor.execute("INSERT INTO users VALUES (?, ?)", ("Neelkanth", 25))
conn.commit()

for row in cursor.execute("SELECT * FROM users"):
    print(row)

conn.close()
```

Summary Table

Module	Best for	Format
pickle	Save/load Python objects	Binary
shelve	Dictionary-like persistent store	Binary
dbm	Simple key-value store	Binary
json	Human-readable, cross-platform	Text (JSON)
sqlite3	Structured queries, relational DB	Binary database


Python Persistent Storage Modules
pickle
shelve
dbm
json
sqlite3
Python objects
Dictionary like
Key-value
Human-readable
Relational database